

ZARZĄDZANIE JAKOŚCIĄ OPROGRAMOWANIA

Software Quality Management

Studium porównawcze: Legacy „Quick & Dirty”
vs. Modern „Production-Ready” Architecture

Autorzy

Marcin Skubiś • Miłosz Rataj • Artur Ścibor

Agenda prezentacji

2 / 18

01

Kontekst i cel projektu

Dlaczego porównujemy dwa podejścia do tych samych wymagań biznesowych

02

Macierz jakości — 9 wymiarów porównania

Architektura, bezpieczeństwo, testy, DevOps, dokumentacja

03

Antywzorce w backend_simple

Dług techniczny i jego konsekwencje na całym cyklu SDLC

04

Best practices w backend_main

Clean Code, DDD, Argon2id, Pydantic, Alembic, testy

05

Analiza kosztów i wartości

Co realnie kosztuje brak jakości i co przynosi inwestycja w jakość

06

Podsumowanie metryczne i wnioski

Liczby, które pokazują różnicę między dwoma światami

Dwa podejścia do tych samych wymagań

3 / 18

CRUD użytkowników + Autoryzacja — identyczny zakres funkcjonalny, dwa światy jakości



Time-to-Market

- Szybkie dostarczenie kosztem długu technicznego
- Wszystko w jednym pliku — minimalna struktura
- Brak walidacji, brak testów, brak migracji
- Hasła w plaintextcie, root w kontenerze
- Działa — dopóki nie pojawi się produkcja



Maintainability

- Architektura warstwowa (DDD / Hexagonal)
- Pydantic, response_model, pełne Swagger /docs
- 13 testów jednostkowych, SQLite in-memory
- Argon2id + pepper, non-root Docker, Alembic
- Gotowe na zespół, skalę i refaktoryzację

Macierz jakości — 9 wymiarów porównania

4 / 18

Cecha	backend_simple	backend_main
Architektura	Monolit spaghetti	DDD / Hexagonal / Layered
Walidacja danych	Brak lub ręczne if/else	Pydantic + silne typowanie
Obsługa błędów	500-tki, stack trace	ApiResponse + kody HTTP
Dokumentacja API	Brak response_model	Swagger UI kompletny pod /docs
Testy automatyczne	0 — niemożliwe (tight coupling)	13 testów, SQLite in-memory
Zarządzanie DB	Hardcoded schema	Alembic — wersjonowane migracje
Bezpieczeństwo	Plaintext, root user	Argon2id + pepper, non-root
Observability	print() w konsoli	Structured JSON logging
DevOps	Single-stage Dockerfile	Multi-stage, healthchecks, .dockerignore

backend_simple — antywzorce i dług techniczny

5 / 18



Brak separacji warstw

HTTP, logika biznesowa i baza danych — wszystko wymieszane w main.py. Każda zmiana wymaga edycji tego samego pliku.



Plaintext hasła w bazie

Hasła zapisane jak nazwiska. Endpoint GET /users/all zwraca je wszystkie. Naruszenie RODO i katastrofa wizerunkowa.



Brak testów = brak refaktoringu

Tight coupling uniemożliwia testy jednostkowe. Każda zmiana to ślepy strzał — regresje wychodzą na produkcji.



Niepowtarzalne buildy

requirements.txt bez wersji + pip install = inny build dziś, inny jutro. „Działa u mnie” staje się jedynym dowodem.



Globalna sesja DB

session = SessionLocal() globalnie. Wyciek pamięci, brak rollback, problemy ujawniają się dopiero pod obciążeniem.



Root user w Dockerze

Aplikacja uruchamiana jako root. Container escape = przejęcie hosta. CORS allow_origins=["*"] dopełnia obrazu.

backend_main — Clean Code i nowoczesne standardy

6 / 18

01



Architektura warstwowa

api/ (kontrolery, schematy) → core/ (handlers + repozytoria) → database/ (modele, migracje). Każda warstwa zna tylko swoją rolę.

02



Bezpieczeństwo by design

Argon2id (zwycięzca PHC 2015) + pepper z env. EmailStr i walidatory na schematach. CORS zawężony do localhost:3000.

03



Testy jednostkowe + fixtures

Warstwa repo pokryta testami (13 plików). SQLite in-memory — CI nie potrzebuje PostgreSQL. pytest + confest.py z fixturami.

04



Alembic — wersjonowane migracje

Każda zmiana schematu = jeden plik w versions/. Rollback jedną komendą. Połączenie: pool_size=20, pre_ping=True.

05



Multi-stage Dockerfile

Non-root user, healthchecks (pg_isready), .dockerignore. docker-compose z condition: service_healthy. env/local.env i env/stg.env.

06



Automatyzacja + uv.lock

pyproject.toml + uv (10–100× szybsze od pip). Makefile z 15+ targetami. Deterministyczne buildy, CI/CD z bramkami jakości.

Architektura — separacja odpowiedzialności

7 / 18

backend_simple

MONOLIT • wszystko w jednym pliku

main.py

FastAPI routing + Pydantic schemas
Logika biznesowa (if/else)
SQLAlchemy queries inline
Hashowanie haseł (brak)
Sesja DB (globalna)
Walidacja wejścia (brak)
Obsługa błędów (brak)
Logging (print)

→ Tight coupling • niemożliwe testy • każda zmiana ryzykuje regresją

backend_main

LAYERED • 3 warstwy + jasny kontrakt

api/

FastAPI controllers, Pydantic schemas, response_model



core/

handlers/ (business logic) + repository/psql/ (DB access)



database/

SQLAlchemy models, sesja, migracje Alembic

→ Kontrakt tuple[data, error, bool] • testy izolowane • refaktoring bez strachu

Bezpieczeństwo — od katastrofy do OWASP 2024

8 / 18

Pięć kategorii podatności, jeden przykład — hasło użytkownika



backend_simple

```
INSERT INTO users (email, password)
VALUES ('jan@example.com', 'MojeHaslo123!');
```

- Plaintext w kolumnie password
- GET /users/all zwraca hasła
- Brak walidacji EmailStr
- CORS allow_origins=["*"]
- Każdy endpoint publiczny

Naruszenie RODO • art. 32 GDPR • kary finansowe



backend_main

```
password = argon2.hash(
    raw + PEPPER # z env
)
# → $argon2id$v=19$m=65536,t=3,p=4$...
```

- Argon2id — zwycięzca PHC 2015
- Pepper z env (poza bazą)
- EmailStr + walidatory Pydantic
- CORS: tylko localhost:3000
- Pole password nigdy nie opuszcza serwera

Zgodność z OWASP Top 10 2024 • fundament pod JWT

Obsługa błędów — od HTTP 500 do semantyki

9 / 18

ZASTANE — wyjątek = stack trace

```
@app.get("/user/{id}")
def get_user(id: int):
    return session.query(User).filter_by(id=id).one()    # NoResultFound → 500 + stack trace
```

NOWE — błąd jako wartość, kod HTTP zgodny z semantyką

```
data, error, ok = repo.find_user(id)
if not ok:
    return error_response(
        type_module="user",
        type_error="not_found",
        http=404,
    )
return ApiSuccessResponse[UserDT0](data=data)
```

404 Not Found

401 Unauthorized

409 Conflict (duplikat)

500 Internal Server Error

backend_simple — pusty /docs

GET /users/all

Response: 200

(brak response_model)

- Brak summary, description, tagów
- Konsumenci nie wiedzą, czego oczekiwać
- Kody błędów niewidoczne w spec
- Duplikacja kodu obsługi błędów
- Frontend integruje się „na ślepo”

backend_main — kompletny /docs

POST /api/v1/user/register

`response_model = ApiSuccessResponse[UserDT0]`

`responses = {409: ApiErrorResponse, 500: ApiErrorResponse}`

- `response_model` na każdym endpointzie
- summary, description, tagi
- Pełna specyfikacja kodów błędów
- Wspólna `error_response()` — bez duplikacji
- Frontend generuje typy z OpenAPI

Testy automatyczne — od 0 do 13 plików

11 / 18

0

testów w backend_simple

pokrycie 0% • refaktoring = loteria

Struktura testów w backend_main

```
tests/  
├── conftest.py          # fixtures: engine, db_session, test_user  
├── core/repository/psql/user/  
│   ├── test_login.py  
│   ├── test_register.py  
│   ├── test_one.py  
│   ├── test_collection.py  
│   ├── test_update.py  
│   └── test_delete.py
```

13

testów w backend_main

~100% warstwy repo • happy + error path



SQLite in-memory

CI bez uruchomionego PostgreSQL



Fixtures pytest

engine (session), db_session
(function), test_user



make test

Jedna komenda, deterministyczny
wynik

Baza danych — od „magicznego schematu” do Alembic

12 / 18

backend_simple

```
Base.metadata.create_all(bind=engine)
```

- Schemat „magicznie” pojawia się przy starcie
- Brak historii zmian
- **Rollback NIEMOŻLIWY**
- Zmiana kolumny = ręczny DROP TABLE na prod
- Globalna sesja DB → wyciek pamięci
- Brak connection pool

backend_main + Alembic

```
alembic upgrade head  
alembic downgrade -1
```

- Każda zmiana = plik w versions/
- Wersjonowanie i odwracalność (downgrade)
- `managed_session()` — context manager + rollback
- `pool_size=20, max_overflow=10, pre_ping=True`
- Audit trail przez Hibernate Envers / audit schema
- Liquibase performance data z kontekstami

Konteneryzacja — single-stage vs multi-stage

13 / 18

backend_simple

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

- Single-stage • USER root (domyślnie)
- Brak .dockerignore — sekrety i .git w obrazie
- Brak healthcheck — orkiestrator nie wie nic
- docker-compose bez service_healthy
- Plik .env wspólny dla wszystkich środowisk

backend_main

```
FROM python:3.11-slim AS builder
RUN uv sync --frozen --no-dev

FROM python:3.11-slim AS runtime
RUN useradd -r app && mkdir /app
USER app
HEALTHCHECK CMD curl -f /health || exit 1
```

- Multi-stage • obraz „slim” z non-root user
- .dockerignore — bez sekretów i .git
- Healthcheck + pg_isready dla PostgreSQL
- docker-compose: condition: service_healthy
- env/local.env, env/stg.env — separacja środowisk

Zależności i CI/CD — deterministyczne buildy

14 / 18

ZARZĄDZANIE ZALEŻNOŚCIAMI

requirements.txt

```
fastapi
sqlalchemy
uvicorn
```

Brak lock • pip install pobierze najnowsze • „działa u mnie”

pyproject.toml + uv.lock

```
[dependencies]
fastapi = "^0.115"
sqlalchemy = "^2.0.34"
# uv.lock – hash każdego pakietu
```

uv sync --frozen • 10–100× szybsze niż pip • deterministyczne

CI/CD I AUTOMATYZACJA

Lint

Ruff

Type check

Mypy

Security

Bandit

Test

pytest + SQLite

Build

Docker multi-stage

Deploy

+ healthcheck

Onboarding nowego developera:

make install && make run

Koszt długu technicznego — co naprawdę kosztuje brak jakości

15 / 18

„Działa na moim komputerze” — sześć faktur, które przychodzą później



Plaintext hasła

Naruszenie danych = katastrofa wizerunkowa, kary RODO (do 4% obrotu rocznego)



Brak migracji

Każda zmiana schematu wymaga ręcznego DROP TABLE na produkcji — downtime i ryzyko utraty danych



Globalna sesja DB

Wyciek pamięci ujawnia się dopiero pod obciążeniem — po wdrożeniu, w nocy, na produkcji



Brak testów

Refaktoryzacja niemożliwa bez ryzyka regresji. Kod zamarza — każdy boi się go dotknąć



Brak response_model

Swagger jest bezużyteczny — konsumenci API nie wiedzą, czego się spodziewać. Integracje wybuchają



Root w Dockerze

Container escape = przejęcie hosta. Jeden CVE i atakujący ma całą maszynę

Wartość inwestycji w jakość — każda decyzja procentuje

16 / 18

Więcej czasu na starcie — wielokrotne oszczędności później



Warstwowa architektura

Nowy developer rozumie strukturę w 15 minut, nie 3 godziny. Każda warstwa testowalna izolowanie



Alembic

Zmiana schematu = jeden plik migracji. Rollback jedną komendą. Pełna historia w git



Argon2id + pepper

Bezpieczeństwo zgodne ze standardami OWASP 2024. Fundament pod JWT i autoryzację ról



Pydantic + response_model

Błędy walidacji wykrywane przed dotarciem do bazy. Swagger generuje pełny kontrakt



Testy jednostkowe

Refaktoryzacja bez strachu. Regresje wykrywane natychmiast. CI nie wymaga PostgreSQL



Makefile + CI/CD

Onboarding = make install && make run. Bramki jakości wyłapią błędy przed mergem

Podsumowanie metryczne — liczby pokazują różnicę

Metryka	backend_simple	backend_main
Liczba plików Python	~6	~50+
Warstwy architektury	1 (monolit)	3 (api, core, database)
Testy automatyczne	0	13 plików (6 operacji × 2-3 testy)
Pokrycie warstwy repo	0%	~100% (happy + error path)
Dokumentacja endpointów	Niekompletna	Pełna (response_model + kody błędów)
Krytyczne podatności	5+	0
Czas onboardingu developera	Wysoki	Niski (make install && make run)
Rollback schematu DB	Nie	Tak (alembic downgrade)
Deterministyczność buildu	Nie	Tak (uv.lock)

Jakość kosztuje na początku — brak jakości kosztuje przez cały cykl życia produktu

WNIOSEK

Jakość to inwestycja, nie koszt.

*Każda dobra decyzja architektoniczna procentuje na każdym etapie SDLC.
Każdy skrót — wraca w postaci faktury.*

DZIĘKUJEMY ZA UWAGĘ • PYTANIA?